TRABAJO DE INVESTIGACIÓN E IMPLEMENTACIÓN · GRUPO 02

TDD y Mutation Testing

Desarrollo guiado por pruebas y evaluación de la efectividad de las pruebas

Integrantes: completar

Asignatura: Ingeniería de Software · Fecha: 3 de noviembre de 2026

Resumen

Este trabajo explica Test-Driven Development (TDD) y Mutation Testing, compara su propósito y presenta un prototipo web autónomo. TDD utiliza ciclos de prueba primero, implementación y refactorización. Mutation Testing modifica deliberadamente el código para verificar si las pruebas descubren esos cambios. Las prácticas se complementan: una ayuda a construir una suite y la otra a evaluar qué tan sensible es frente a errores sencillos.

1. TDD: desarrollo guiado por pruebas

TDD (Test-Driven Development) es una práctica de desarrollo en la que se escribe primero una prueba automatizada que describe el siguiente comportamiento requerido. Después se implementa el código mínimo para satisfacerla y se reorganiza el diseño sin cambiar el comportamiento. El ciclo se conoce como Red-Green-Refactor. Fowler también destaca la importancia de listar y ordenar los casos antes de recorrer los ciclos.

RED	GREEN	REFACTOR
Crear una prueba nueva y observar que falla por el comportamiento esperado.	Escribir el mínimo código para que pase y ejecutar toda la suite.	Mejorar nombres, estructura y duplicación sin alterar el comportamiento.

¿Para qué sirve y cuándo se usa?

TDD ofrece feedback temprano, hace explícitas las expectativas y aporta una red de regresión durante cambios y refactorizaciones. Se usa al implementar reglas de negocio, lógica con escenarios y límites conocidos, y cuando el costo de una regresión es alto. Requiere casos útiles: escribir pruebas primero por sí solo no garantiza que cubran todas las reglas.

Ventajas y contras

BENEFICIOS	COSTOS Y LÍMITES
• Aclara el resultado esperado antes de codificar. • Reduce ciclos largos de detección de errores. • Facilita cambios pequeños y refactorización. • Mantiene ejemplos ejecutables del comportamiento.	• Añade esfuerzo inicial y requiere disciplina. • Pruebas mal diseñadas pueden reflejar requisitos equivocados. • No sustituye pruebas de integración, exploratorias ni revisión. • Aserciones frágiles elevan el costo de mantenimiento.

2. Mutation Testing: evaluar las pruebas

Mutation Testing es una técnica que crea variantes defectuosas del código mediante pequeños cambios llamados mutaciones. La suite existente se ejecuta contra cada variante. Si al menos una prueba falla, el mutante queda muerto (detectado); si todas pasan, sobrevive y puede revelar una prueba faltante o una aserción débil. También existen mutantes sin cobertura, errores de ejecución y mutantes equivalentes que no cambian el comportamiento observable.

Métrica

Puntaje de mutación = mutantes detectados ÷ mutantes válidos × 100. Stryker clasifica estados como killed, survived, no coverage y timeout; errores de compilación o ejecución se informan aparte. Debe leerse junto con la cobertura y la revisión de los casos: el porcentaje no prueba que el sistema esté libre de defectos ni identifica todos los errores posibles.

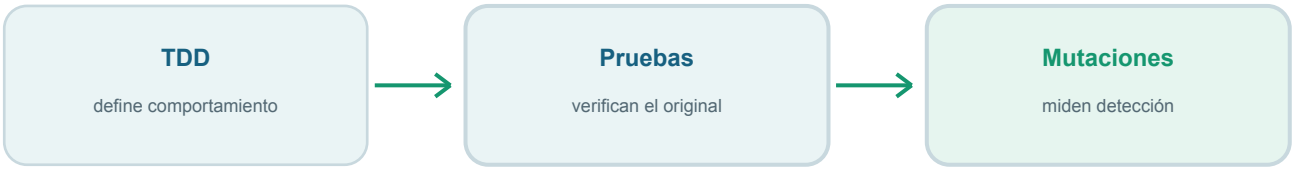
¿Cuándo se usa?

Después de contar con pruebas automatizadas y, en particular, en lógica determinista donde las mutaciones pueden ejecutarse con costo razonable. Es útil para priorizar dónde mejorar la suite. Puede ser lento en proyectos grandes porque ejecuta muchas variantes; se controla limitando archivos, operadores o ejecutando análisis selectivos en CI.

Beneficios y límites

Aporta	No garantiza
• Evidencia más fuerte que cobertura de líneas por sí sola. • Detecta pruebas que ejecutan código pero no verifican su resultado. • Sugiere casos límite concretos para añadir.	• Ausencia de defectos en producción. • Calidad del requisito o de datos de prueba. • Equivalencia de mutantes o buen comportamiento fuera de los cambios generados. • Rentabilidad para cualquier módulo.

Relación entre las prácticas



TDD impulsa la construcción incremental de pruebas. Mutation Testing puede evaluarlas luego. No son técnicas excluyentes ni el puntaje de mutación reemplaza el diseño cuidadoso de casos.

3. Prototipo implementado

El prototipo del repositorio está en `tdd-mutation-testing/index.html`. Se abre directamente en el navegador, sin instalación. Su ejemplo es una función de descuento y tres aserciones: 10 % de 100 debe dar 90; 0 % de 80 debe dar 80; y 20 % de 50 debe dar 40.

Variantes del laboratorio

VARIANTE	CAMBIO SIMULADO	RESULTADO ESPERADO
Original	precio × (1 - tasa / 100)	3 de 3 pruebas pasan
M1 · Ignorar	Devuelve el precio sin descuento	Las pruebas con tasa positiva fallan
M2 · Sumar	Suma la tasa al precio	Las pruebas con tasa positiva fallan
M3 · Base incorrecta	Divide precio por (100 - tasa)	Las pruebas observan resultado incorrecto

Cómo usarlo

1. Abre `tdd-mutation-testing/index.html`.
2. Ejecuta el código original y confirma que pasa 3/3.
3. Selecciona y ejecuta cada mutación M1, M2 y M3.
4. Al evaluar las tres, consulta el puntaje y el conteo de mutantes muertos y sobrevivientes.

Resultado esperado: 3 mutantes muertos, ninguno sobreviviente y puntaje del prototipo de 100 %. El puntaje se calcula con tres mutantes válidos. La aplicación es un modelo educativo, no sustituye un motor como Stryker ni implementa su conjunto completo de estados.

Cierre

TDD ayuda a que el comportamiento sea comprobable desde el comienzo. Mutation Testing comprueba si una suite detecta errores sencillos y anima a reforzar pruebas importantes. Usadas con criterio, ambas prácticas mejoran la retroalimentación; su efectividad depende de requisitos correctos, ejemplos representativos y revisión humana.

4. Referencias

Fuentes consultadas el 3 de noviembre de 2026

1. Martin Fowler, “Test-Driven Development”. Explica el ciclo y la preparación de casos de prueba.
https://martinfowler.com/bliki/TestDrivenDevelopment.html

2. Stryker Mutator, documentación oficial. Estados de los mutantes y configuración de la herramienta.
https://stryker-mutator.io/docs/

3. Stryker Mutator, “Mutant states and metrics”. Definiciones de killed, survived, no coverage, timeout y puntuación.
https://stryker-mutator.io/docs/mutation-testing-elements/mutant-states-and-metrics/

4. Martin Fowler, “Workflows of Refactoring”. Describe el patrón Red-Green-Refactor.
https://martinfowler.com/articles/workflowsOfRefactoring/

Material complementario

Prototipo y guía de ejecución: carpeta `tdd-mutation-testing/` en el repositorio. Antes de exponer, completar los nombres de integrantes del Grupo 02.